

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА

по теме

Разработка и документирование требований к учебному проекту

Выполнил
студент гр. 250541

Бойцов А.В.

Проверил
ассистент каф. ЭВМ

Богдан Е.В.

Минск 2025

1 ВВЕДЕНИЕ

Монополия» — программное приложение, реализующее классическую настольную экономическую игру в цифровом формате. Система предназначена для обучения основам стратегического мышления, управления ресурсами и принятия решений в условиях ограничений. Приложение позволяет пользователям взаимодействовать с игровым полем, совершать сделки, управлять собственностью и соревноваться за доминирование на рынке.

Основная идея проекта — создать интерактивную, расширяемую и структурированную цифровую версию игры, построенную на строгой архитектуре, определённой UML-диаграммами, и полностью соответствующую требованиям, описанным в SRS. Приложение реализует ключевые механики оригинальной игры, включая покупку и продажу собственности, начисление ренты, случайные события и определение победителя.

Первая версия приложения включает следующий функционал:

- инициализация игрового поля согласно правилам; – создание игроков и управление их состоянием (баланс, собственность, позиция на поле);
- обработка хода игрока: бросок кубиков, перемещение, выполнение действий клетки;
- покупка, продажа и улучшение собственности; – начисление и получение ренты;
- обработка специальных клеток (Шанс, Общественная казна, Налог, Тюрьма);
- графический интерфейс для отображения состояния игры;
- базовый игровой ИИ (опционально, если предусмотрено SRS).

В рамках текущей версии проекта **не предусматривается** реализация следующих возможностей:

- сетевой многопользовательский режим;
- сохранение и загрузка состояния игры;
- расширенная экономическая модель (динамические цены, инфляция, аукционы);
- сложный ИИ с обучением или прогнозированием;
- модульное редактирование правил игры пользователем.

2 ТРЕБОВАНИЯ ПОЛЬЗОВАТЕЛЯ

2.1 Программные интерфейсы

Приложение должно предоставлять пользователю программные интерфейсы, обеспечивающие взаимодействие с игровым процессом и визуальными элементами. Интерфейсы включают:

- графический интерфейс игрового поля;
- элементы управления ходом игрока (бросок кубиков, покупка, строительство, завершение хода);
- диалоговые окна для подтверждения действий;
- систему уведомлений и сообщений об ошибках;
- интерфейс отображения состояния игроков и собственности.

Все интерфейсы должны быть интуитивно понятными, устойчивыми к ошибкам пользователя и соответствовать выбранному Style Guide.

2.2 Интерфейс пользователя

2.2.1 Общая структура приложения

Приложение должно включать следующие основные экраны и элементы:

- главное окно приложения;
- игровое поле с клетками;
- панель информации о текущем игроке;
- панель действий;
- всплывающие окна событий.

Структура интерфейса должна обеспечивать доступ ко всем игровым функциям без скрытых меню.

2.2.2 Главное меню

Главное меню должно предоставлять пользователю:

- выбор количества игроков;
- доступ к правилам;
- настройки (если предусмотрено SRS);
- выход из приложения.

Макет запуска игры представлен в виде диалоговых окон выбора количества игроков и введении никнеймов на рисунках 1 и 2.

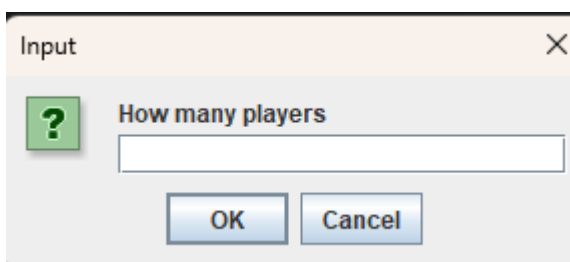


Рисунок 1 – Макет окна «Выбор количества игроков»



Рисунок 2 – Макет окна «Ввод имени игрока»

Выход из приложения происходит при нажатии кнопки «X».

2.2.3 Игровое поле

Игровое поле должно отображать:

- клетки с названиями и типами;
- позиции игроков;
- стоимость собственности;
- визуальную подсветку доступных действий;
- информацию о текущем ходе.

Пользователь должен иметь возможность наблюдать за изменениями состояния поля в реальном времени.

Макет окна «Игровое поле» представлен на рисунке 3.

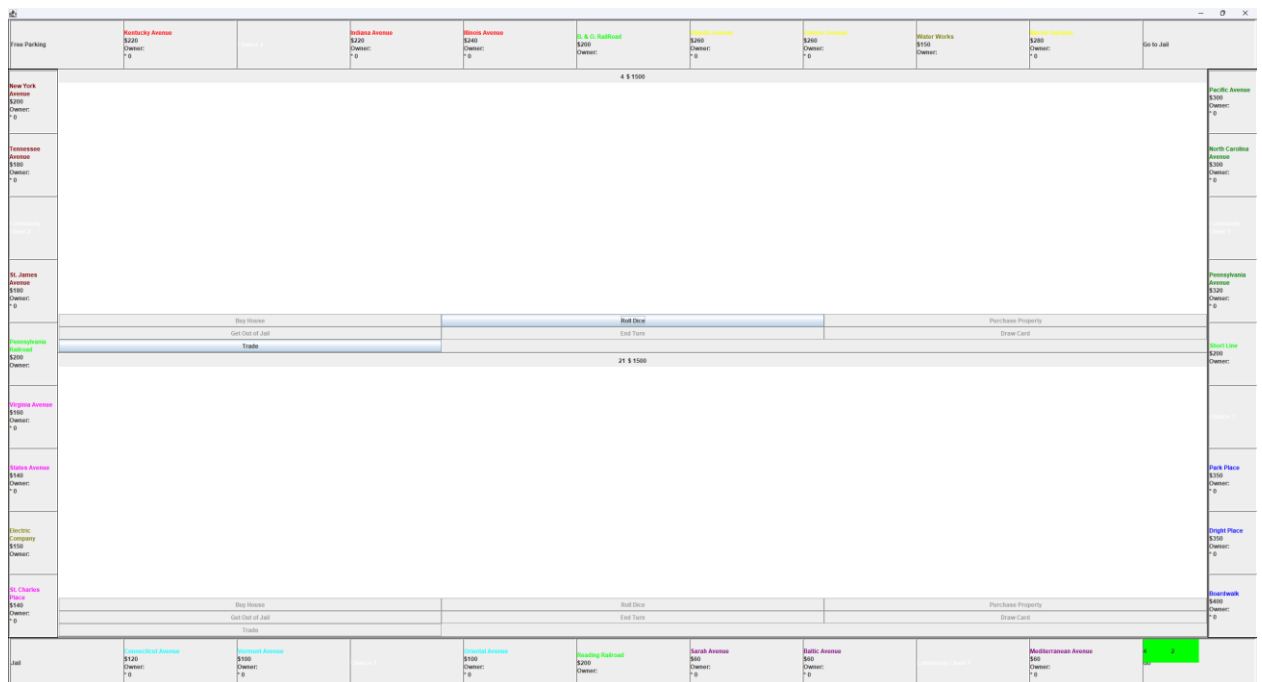


Рисунок 3 – Макет окна «Игровое поле»

2.2.4 Окно игрока

Окно игрока должно содержать:

- текущий баланс;
- список собственности;
- количество домов/отелей;
- доступные действия (покупка, строительство, продажа, пропуск хода).

Макет окна игрока представлен на рисунке 4.

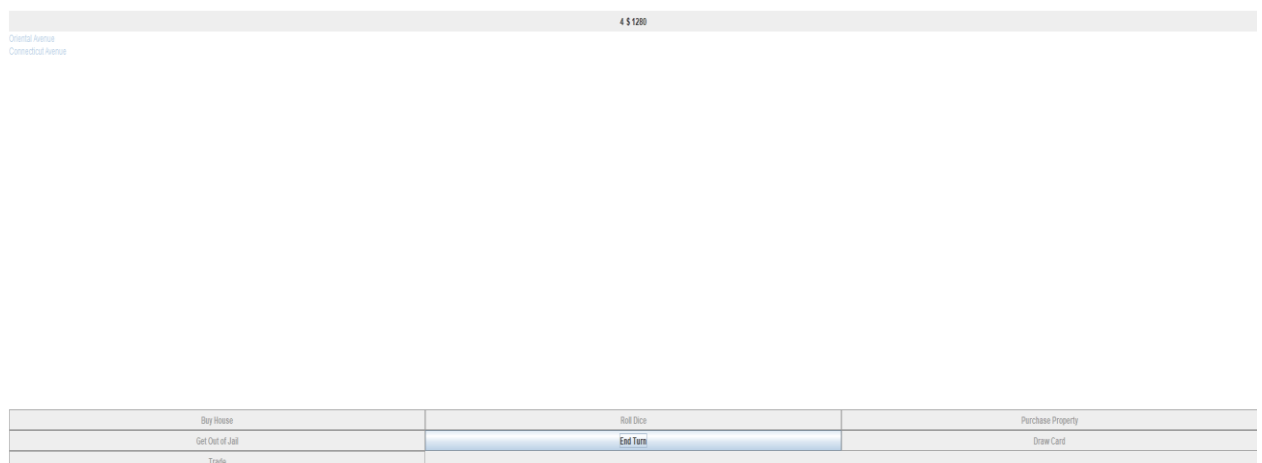


Рисунок 4 – Макет окна игрока

2.2.5 Диалоговые окна действий

Приложение должно отображать диалоговые окна для:

- покупки собственности;
- строительства домов и отелей;
- оплаты ренты;
- выполнения случайных событий;
- подтверждения критических действий.

Диалоговые окна должны содержать краткое описание действия и варианты выбора.

2.2.6 Взаимодействие между экранами

Приложение должно обеспечивать:

- переход между главным меню и игровым полем;
- отображение всплывающих окон поверх игрового поля;
- обновление интерфейса после каждого действия игрока;

2.3 Характеристики пользователей

2.3.1 Целевая аудитория

Игра монополия предназначена для любых пользователей приложения желающих сыграть в игру и знакомых с настольной игрой “Монополия”. Количество игроков может варьироваться от 1 до 8 пользователей в зависимости от потребностей.

2.3.2 Роли пользователей

В приложении «Монополия» предусмотрены несколько типов пользователей, отличающихся уровнем взаимодействия с системой, набором доступных действий и степенью влияния на игровой процесс. Каждая роль определяет, какие функции пользователь может выполнять, какие интерфейсы доступны и какие ограничения накладываются на его действия.

1. Функции игрока:

- участие в игровой сессии;
- выполнение хода (бросок кубиков, перемещение по полю);
- принятие решений по покупке собственности;
- строительство домов и отелей;
- оплата ренты и налогов;
- выполнение действий по карточкам «Шанс» и «Общественная казна»;
- управление своим балансом;
- принятие решений об обмене собственности, продаже.

- взаимодействие с диалоговыми окнами и подсказками.

Ограничения игрока:

- количество игроков ограничено правилами игры;
- игрок не может выполнять действия вне своего хода;
- игрок не может нарушать правила, определённые игровой логикой.

2. Наблюдатель — опциональная роль

Роль наблюдателя используется в учебных, демонстрационных или тестовых сценариях. Наблюдатель не участвует в игре, но имеет доступ к просмотру состояния игрового процесса.

Функции наблюдателя:

- просмотр игрового поля;
- наблюдение за ходами игроков;
- просмотр балансов и собственности игроков;
- анализ игровой ситуации.

Ограничения наблюдателя:

- отсутствие возможности выполнять игровые действия;
- невозможность влиять на ход игры;
- доступ только к режиму чтения.

2.3.3 Общие характеристики всех пользователей

Пользователи приложения «Монополия» обладают рядом общих характеристик, определяющих их возможности, уровень подготовки и предполагаемое взаимодействие с системой. Эти характеристики учитываются при проектировании интерфейса, выборе механик взаимодействия и определении требований к удобству использования.

1. Пользователи должны обладать минимальными техническими навыками, включая:

- умение запускать настольные приложения;
- способность работать с мышью и клавиатурой;
- понимание базовых элементов интерфейса (кнопки, окна, диалоговые сообщения);
- умение читать и интерпретировать визуальную информацию.

Эти навыки считаются достаточными для полноценного взаимодействия с приложением.

2. Пользователь должен иметь базовое представление о правилах настольной игры «Монополия», включая:

- назначение игрового поля и клеток;
- принципы покупки собственности;
- начисление ренты;

- условия банкротства;
- последовательность хода.

Глубокое знание правил не требуется, так как интерфейс предоставляет подсказки и визуальные пояснения.

3.Отсутствие необходимости технических знаний

Для использования приложения не требуется:

- знание программирования;
- понимание внутренней архитектуры игры;
- работа с файлами конфигурации;
- использование командной строки.

Все взаимодействие осуществляется через графический интерфейс.

2.4 Предположения и зависимости

2.4.1 Предположения

В рамках разработки и эксплуатации приложения «Монополия» предполагается, что:

- Пользователь знаком с базовыми правилами игры Пользователь понимает основные механики: бросок кубиков, покупка собственности, рента, банкротство.
- Приложение запускается на поддерживаемой платформе Устройство пользователя соответствует минимальным системным требованиям (ОС, JRE, разрешение экрана).
- Пользователь использует стандартные устройства ввода предполагается наличие мыши и клавиатуры или аналогичных средств управления.
- Количество игроков соответствует правилам. Минимальное и максимальное число участников определяется SRS (обычно 2–4).
- Пользователь не пытается нарушать правила игры Вводимые данные и действия соответствуют игровым ограничениям.
- Игровой процесс проходит последовательно. Игроки совершают ходы по очереди, без параллельных действий.
- Пользователь обладает базовыми навыками работы с ПК. Это необходимо для корректного взаимодействия с интерфейсом.

2.4.2 Зависимости

Работа приложения зависит от ряда внешних факторов, которые не контролируются системой напрямую:

- **Java Runtime Environment** Приложение требует установленной версии JRE, соответствующей требованиям проекта.
- **Графическая библиотека (Swing или JavaFX)** Корректная работа интерфейса зависит от выбранной технологии отображения.
- **Операционная система** Приложение должно запускаться на ОС, поддерживающих JRE (Windows, Linux, macOS).
- **Разрешение экрана** Интерфейс корректно отображается только при минимальном поддерживаемом разрешении.
- **Стабильность среды выполнения** Система должна работать без критических ошибок ОС или сторонних программ.

3 СИСТЕМНЫЕ ТРЕБОВАНИЯ

3.1 Функциональные требования

3.1.1 Инициализация игры

Система должна обеспечивать:

- создание нового игрового сеанса;
- выбор количества игроков;
- инициализацию игрового поля согласно правилам;
- распределение стартового капитала.

3.1.2 Управление игровым процессом

Система должна предоставлять функциональность:

- выполнения хода игрока;
- генерации результата броска кубиков;
- перемещения фишки игрока по полю;
- выполнения действий клетки (покупка, рента, налог, событие).

3.1.3 Управление собственностью

Система должна поддерживать:

- покупку собственности;
- оплату ренты;
- строительство домов и отелей;
- продажу собственности;
- передачу собственности при банкротстве.

3.1.4 Обработка событий

Система должна корректно обрабатывать:

- карточки «Шанс»;
- карточки «Общественная казна»;
- попадание в тюрьму;
- освобождение из тюрьмы;
- налоговые клетки.

3.1.5 Определение состояния игры

Система должна:

- отслеживать баланс игроков;
- определять банкротство;
- завершать игру при наличии единственного игрока;
- отображать победителя.

3.1.6 Взаимодействие с пользователем

Система должна:

- отображать текущее состояние игры;
- предоставлять доступные действия;
- выводить уведомления и ошибки;
- обеспечивать корректную реакцию на ввод пользователя.

3.2 Нефункциональные требования

3.2.1 Требования к производительности

- Время отклика интерфейса не должно превышать 200 мс.
- Обработка хода игрока должна выполняться менее чем за 1 секунду.
- Приложение должно корректно работать при максимальном количестве игроков, указанном в SRS.

3.2.2 Требования к надёжности

- Приложение должно корректно обрабатывать некорректный ввод пользователя.
- Ошибки должны сопровождаться понятными уведомлениями.
- Система не должна аварийно завершаться при стандартных игровых сценариях.

3.2.3 Требования к удобству использования

- Интерфейс должен быть интуитивно понятным.
- Все основные действия должны быть доступны в 1–2 клика.
- Визуальные элементы должны быть читаемыми и контрастными.

3.2.4 Требования к совместимости

- Приложение должно работать на ОС, поддерживающих Java Runtime Environment.
- Интерфейс должен корректно отображаться на экранах с различным разрешением.
- Приложение должно быть совместимо с выбранной графической библиотекой (Swing/JavaFX).

3.2.5 Требования к безопасности

- Приложение не должно выполнять опасных операций с файловой системой.
- Данные игроков должны храниться только в рамках игрового сеанса (если нет функции сохранения).

3.2.6 Требования к поддерживаемости

- Код должен соответствовать Style Guide.
- Архитектура должна быть модульной и расширяемой.
- Классы и методы должны быть документированы (Javadoc).
- Структура пакетов должна соответствовать UML-диаграммам.